Three Member Presentation Script

Timing target: approximately 13.5 minutes, including short diagram pauses and demonstration actions. Each member presents six slides and approximately 4.5 minutes. Rehearse at 130–145 words per minute; timing is an estimate, not a measured rehearsal.

Slide 1 CodeTrace AI

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

Good morning. We are Yash Raj Agarwal, Nishaanth S and Aman Sharma. Our project is titled Stylometric and Structural Classification Framework for Detecting LLM-Generated Code Submissions. We built CodeTrace AI as a local machine learning demonstration. It extracts features from Python source without running the program, then uses a saved classifier to estimate which labeled style category the sample resembles. We will explain the implementation, show its measured results, and demonstrate where it is fragile. We want to be clear from the beginning: the included data is synthetic, so this presentation does not claim validated authorship detection.

Slide 2 Problem statement

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

The question we explore is whether measurable code style and structure can distinguish labeled examples. Code differs from ordinary prose: assignments specify algorithms, libraries introduce standard expressions, and programmers often produce similar valid solutions. A student can also add comments or rename identifiers without changing the program. These facts make the relationship between style and origin uncertain. Academic integrity motivates the question, but a probability alone cannot determine whether someone violated a policy. Our scope is to study a transparent classification pipeline and examine its failure modes, rather than automate academic decisions.

Slide 3 Motivation and objectives

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

We selected a compact design that we can explain from source code to prediction. Our first objective is to turn code into numerical features using static analysis. Our second is to compare Logistic Regression, a Decision Tree and a Random Forest under the same data protocol. Our third is to connect the selected saved pipeline to a website and test how its outputs respond to controlled changes. The resulting contribution is an educational system with reproducible experiments. A stronger authorship study would require verified data, independent task families and a more rigorous evaluation protocol, which we discuss later.

Slide 4 Literature and research gap

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

Our literature record separates direct evidence from background. Reference one is a C# code-stylometry study. Reference two examines text detector reliability and therefore motivates caution rather than supplying code-detection accuracy. Reference three provides source-code authorship comparison context, while reference four is the Random Forest foundation. We did not transfer their numerical findings into our own results. The accessible review did not verify twenty to twenty-five qualifying indexed journal articles, and the report states that gap explicitly. The immediate gap for our project is also empirical: we need a provenance-verified, task-diverse dataset before claiming real authorship performance.

Slide 5 System architecture

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Trace the editable architecture arrows.

Follow the diagram from left to right. The React client sends source code and the selected Python language as JSON. FastAPI validates the request length, language and syntax. The backend extracts static features and constructs the one-row input expected by the saved preprocessing pipeline. TF-IDF transforms the lexical part and StandardScaler transforms the numerical part. The selected classifier returns class probabilities. The backend finds the AI class by name and returns the estimate plus feature summaries. The source never executes, and no external language-model API is called. Training is a separate offline process, shown below the inference path.

Slide 6 Dataset and partitions

Speaker: YASH RAJ AGARWAL. Approximate slot: 45 seconds including visual cues.

Cue: Point to the actual data or diagram while explaining.

This slide is essential for interpreting every score. The CSV contains two hundred synthetic records generated from a few code templates. Cleanup removes forty duplicate rows, leaving one hundred and sixty unique examples, eighty per label. Group-based splitting produces ninety-six training, thirty-two validation and thirty-two test examples. The groups are disjoint by identifier, but they are not twenty independent algorithmic tasks. Shared template families remain across partitions, and the labels describe designed style categories rather than actual human or LLM provenance. I will now hand over to Nishaanth to explain exactly how those code samples become machine learning inputs.

Slide 7 Feature engineering

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

The numeric extractor combines style, structure and size measurements. It counts identifiers by occurrence and computes their mean length and naming ratios. It measures line length, comments and blank lines. It uses Python AST nodes to count functions, loops, conditionals, classes, exceptions and docstrings, together with a selected-construct nesting measure. Finally, it records source length, token count and unique-token ratio. This is eighteen numerical features in total. Naming entropy and cyclomatic complexity are not in the implementation. We explain them as possible future features, but do not present them as measured outputs. TF-IDF adds separate lexical coordinates.

Slide 8 Mathematical foundations

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

The equations here match the code. Sublinear term frequency dampens repeated words, and smoothed inverse-document frequency increases the weight of less common terms. Each lexical vector is then L2-normalized. Numerical features use training means and standard deviations. Logistic Regression maps a weighted score through the sigmoid. For example, a score of zero gives probability one half, while a score of zero point six gives about zero point six four six. The tree uses Gini impurity by default, not an entropy feature. The inference threshold is one half. The confidence displayed by the website is an uncalibrated model output, not an authorship guarantee.

Slide 9 Three implemented models

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

Logistic Regression gives a compact weighted baseline over the combined transformed feature vector. The standalone Decision Tree forms feature-based branches, constrained to depth twelve and at least two samples per leaf. Random Forest averages one hundred and sixty randomized trees with a minimum leaf size of two. All three use balanced class weights and seed forty-two. Shared numeric scaling simplifies the pipeline, although trees do not require it. We did not implement XGBoost or CodeBERT. Those were proposed in the original synopsis, and should be described as possible future extensions if a faculty member asks about them.

Slide 10 Training and model selection

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

The first grouped split reserves test groups, and the second reserves validation groups from the remainder. Each candidate fits its own transformation and classifier on training examples only. Validation F1, with AI as the positive class, is the main selection criterion. Accuracy breaks an F1 tie, and insertion order resolves a remaining tie. On this fixture all three candidates tie, so Logistic Regression is selected because it is listed first. The saved model remains fitted on the training subset. There is no cross-validation, grid search or later train-plus-validation refit in production. The separate experiment runner compares configurations without overwriting the deployed model.

Slide 11 Actual model comparison

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Point to the actual data or diagram while explaining.

The table reports the real saved evaluation output. Every candidate scores one point zero on accuracy, precision, recall and F1 on the thirty-two test examples. The selected matrix contains sixteen correctly predicted human-style and sixteen correctly predicted AI-style records. There are no errors on this test fixture. That is a limited observation, not a universal performance claim. The generator deliberately repeats strong style cues such as docstrings, and related templates occur across group partitions. Therefore the apparent perfection reflects an easy artificial task. We cannot infer the error rate on actual student submissions or code from a new model.

Slide 12 Explainability and its limits

Speaker: NISHAANTH S. Approximate slot: 45 seconds including visual cues.

Cue: Point to the actual data or diagram while explaining.

The global importance chart sorts absolute coefficients for the selected Logistic Regression model. Code length, docstring count and identifier length rank prominently in this fitted fixture. Absolute values lose direction, and correlated measurements can share predictive weight. The sample feature panel is even more limited: it describes the submitted code but is not a complete local explanation of the prediction. We have not implemented SHAP or permutation importance. Renaming can change several measurements simultaneously, so one probability change does not identify one causal feature. I will now hand over to Aman for the working application, controlled demonstrations and conclusions.

Slide 13 Website demonstration

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Switch between the running views; use screenshots if needed.

The screenshots are from our running application. In the live demonstration I will paste valid Python and press Analyze. The request goes to FastAPI, which loads the saved pipeline and returns the prediction. I will point to the AI-class probability and the extracted structural and style signals. I will then open Model Performance to show the actual saved comparison table and confusion matrix, and About to show the scope and limitations. The interface has three views rather than separate landing and prediction systems. If the server is unavailable, these screenshots and recorded JSON outputs provide an honest demonstration fallback.

Slide 14 Controlled equivalent-code edits

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Point to the actual data or diagram while explaining.

The baseline function filters even integers. We apply several changes while preserving that behavior for ordinary finite integer lists. Long camelCase names change identifier statistics, code length and lexical words. Adding comments changes comment ratio and lexical content. Compact spacing changes line and source length while much AST structure remains unchanged. Replacing the loop with a comprehension changes the selected AST counts. The exact before-and-after probabilities are recorded in the experiment CSV and shown here. Some changes alter the predicted class and others only alter probability. We did not force any label switch; unchanged decisions are meaningful observations too.

Slide 15 Configuration experiments

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Point to the actual data or diagram while explaining.

We also change the learning configuration reproducibly. Stronger and weaker regularization change the baseline probability while all tested validation class metrics remain perfect. Ablation retrains after removing defined feature groups. Those groups overlap conceptually with remaining length and token statistics, so the results are conditional comparisons rather than clean isolation of every notion of style. The tested thresholds leave validation precision and recall unchanged because no validation scores cross the boundaries. Finally, a factorial function probes a different task. We can report its output, but because it has no verified authorship label, we cannot turn that single probe into a generalization accuracy.

Slide 16 Limitations and future scope

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

The strongest limitation is the dataset, not the choice of classifier. Synthetic labels and repeated templates make the measured problem much easier than real authorship detection. Probability outputs are uncalibrated. Equivalent refactoring can change estimates, and there is no evidence of performance on unseen generators, languages or real assignments. Our first improvement should be consented human submissions and matched generated outputs with known provenance. We should split by meaningful task families and authors where appropriate, repeat grouped evaluation and keep an independent final test set. Only then should we compare advanced models, richer AST features, entropy or extra languages.

Slide 17 Conclusion

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

We have completed the computational workflow from source input through static features and fitted transformations to saved-model inference and a working website. We have also made the results and sensitivity experiments reproducible. The experiments show both stable and changing predictions under controlled modifications, which helps explain the relationship between features and learned behavior. The actual claim we can defend is an educational prototype with transparent limitations. We cannot defend operational authorship reliability from this dataset. That distinction is central to the project, because a clear explanation of what the experiment establishes is more valuable than an impressive-looking accuracy number.

Slide 18 References and questions

Speaker: AMAN SHARMA. Approximate slot: 45 seconds including visual cues.

Cue: Keep eye contact; use the slide as a visual anchor.

Thank you. We are ready to discuss the mathematics, source code, data protocol and reliability concerns. Yash can explain the motivation, literature and dataset. Nishaanth can walk through tokenization, AST measurements, TF-IDF, scaling and the classifiers. I can demonstrate the API, saved inference, evaluation and experiments. If a question asks why we chose XGBoost, the accurate answer is that it is not implemented. If asked why the scores are perfect, we will explain the repeated synthetic templates. The references and reproduction scripts are available in the evaluation package for inspection.

Pronunciation and transitions

TF-IDF: “tee eff eye dee eff”; AST: “ay ess tee”; Gini: “jee-nee”; sigmoid: “sig-moyd”; scikit-learn: “sigh-kit learn”; Pydantic: “pie-dan-tick”; ROC AUC: “rock ay-you-see”. The transitions are included at the ends of slides 6 and 12. Do not read every equation symbol; define its terms and explain a small example.

30 general project questions

Q1 What does this project actually classify?

Synthetic code-style labels. The generator assigns human and ai strings to templates. Neither label proves actual provenance.

Q2 What is supervised learning?

Learning an input-to-target relationship from labeled examples. Code features are inputs; the CSV label supplies the training target.

Q3 Why is this binary classification?

There are two target classes. The decision threshold maps a probability score to one of human and ai.

Q4 Why is Logistic Regression called regression?

The model regresses log-odds on features. Its output supports binary classification after a sigmoid and threshold.

Q5 What is a feature?

A numerical measurement of the input. For example, comment-token count divided by code-line count describes a source sample.

Q6 What is a label?

The target category assigned to a training sample. It must be supported by collection provenance in real research; ours is a synthetic fixture label.

Q7 What is training?

Fitting transformations and classifier parameters. Vocabulary, IDF, scaler parameters and model coefficients use only training data.

Q8 Why use validation?

To make model choices before test evaluation. The trainer selects maximum validation F1, then accuracy, with a deterministic order tie break.

Q9 What does testing measure?

Performance on an untouched evaluation partition. It estimates only the population represented by that partition and protocol.

Q10 What is overfitting?

Learning patterns that do not generalize. Repeated generator templates can make held-out records easy even with group-disjoint IDs.

Q11 What is underfitting?

A model is too restrictive to capture useful relationships. Strong regularization may flatten probabilities, although current validation classes remain separable.

Q12 Explain bias and variance.

Bias is systematic model error; variance is sensitivity to training samples. Deeper trees can reduce bias while increasing variance; averaging trees can reduce variance.

Q13 What is domain shift?

The input distribution changes between training and use. Real programs or different algorithms need not resemble the filtering templates.

Q14 Why is accuracy alone insufficient?

It hides error types and depends on class prevalence. A constant human predictor can be accurate if AI examples are rare.

Q15 What is data leakage?

Evaluation information affects fitting or selection. Putting near-identical task variants in different partitions can reward memorization.

Q16 Is the dataset balanced?

After cleanup, yes: 80 examples per label. This is an artificial fixture distribution, not measured educational prevalence.

Q17 How many records are used?

160 unique examples from 200 rows. Forty exact duplicates are removed after stripping boundary whitespace.

Q18 How are labels encoded?

They remain strings handled by scikit-learn. Inference finds the ai class index explicitly rather than assuming a fixed column.

Q19 How are missing values handled?

Missing code or label records are removed. Missing problem IDs get per-row fallback groups, weakening protection against shared-task leakage.

Q20 How are conflicting labels handled?

Training now raises a clear ValueError. An identical program should not silently keep whichever contradictory label appears first.

Q21 How are duplicates detected?

Exact code-string equality after stripping. Renamed or reformatted semantic clones remain undetected.

Q22 What is group splitting?

Keeping related records in one partition. GroupShuffleSplit keeps each problem_id exclusively in training, validation or test.

Q23 Do we use stratified splitting?

No. The grouped split happens to be balanced on this fixture; the trainer now checks both labels exist in each partition.

Q24 Do we use cross-validation?

No in production. A stronger study should repeat group-aware validation across independent task families.

Q25 What are the split sizes?

96 train, 32 validation and 32 test. That corresponds to 12, four and four groups for this fixture.

Q26 Are there 20 independent tasks?

No. The IDs reuse a few filter templates with naming and modulus variants.

Q27 How does tokenization work?

Python tokenize yields lexical tokens. The numeric token statistics include names, operators and literals while excluding comments and layout.

Q28 What is an AST?

A tree of abstract syntax constructs. It captures functions, conditionals and calls while omitting much concrete punctuation.

Q29 How does an AST differ from a parse tree?

It removes grammar details that are not needed for its abstraction. A concrete parse tree tends to retain punctuation and grammar productions; AST nodes represent language constructs.

Q30 What does AST node count include?

All nodes from ast.walk. That includes context nodes such as Load, so it is not merely the number of statements.

20 ML-specific questions

Q31 Does nesting depth mean raw AST depth?

No. It increments selected control constructs and function definitions; expression layers alone do not increment it.

Q32 Are comprehensions counted as for loops?

No. The current loop count checks ast.For, ast.AsyncFor and ast.While, not ast.comprehension.

Q33 What is comment-to-code ratio?

Comment-token count divided by nonblank non-comment-only line count. The denominator is bounded below by one; inline comments count, whereas docstrings are string literals.

Q34 Does the snake-case feature require underscores?

No. Its regex also accepts plain lowercase identifiers and some digits after the initial letter.

Q35 Are identifiers counted once each?

No, by occurrence. Repeated uses affect average identifier length and naming-style ratios.

Q36 What is naming entropy?

Entropy of a specified identifier or character distribution. $H=-\Sigma p \log_2 p$; it is not implemented and must not be claimed as a measured feature.

Q37 What is the entropy of two equally likely symbols?

One bit. $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$.

Q38 What is cyclomatic complexity?

A control-flow measure of independent paths. A common graph expression is $E-N+2P$; it is not computed by this project.

Q39 Why scale numerical features?

To put differently sized measurements on comparable standardized scales. The training mean and population variance define later transformations.

Q40 What happens for a constant feature?

StandardScaler handles zero variance safely. It effectively avoids division by zero; a constant centered value contributes no varying numeric signal.

Q41 What is feature selection here?

No separate supervised selector is implemented. The 1,200-term vocabulary cap is a frequency-based vectorizer limit.

Q42 What is TF-IDF?

A weighted lexical representation using term frequency and document frequency. Rare terms receive larger IDF; per-document L2 normalization follows weighting.

Q43 What is the actual TF formula?

$1+\ln(\text{count})$ for positive counts. sublinear_tf=True dampens repeated-token growth.

Q44 What is the smoothed IDF formula?

$1+\ln((1+N)/(1+df))$. The extra ones avoid zero divisions and follow TfidfVectorizer defaults.

Q45 What are n-grams?

Short adjacent token sequences. The model uses unigram and bigram word features, not full token-order semantics.

Q46 Does TF-IDF retain case?

No, default lowercasing is used. The numeric identifier features still inspect original case and can detect camelCase patterns.

Q47 What happens to an unseen word?

It has no fitted vocabulary column. It adds no lexical contribution, while numeric source features can still change.

Q48 Why are sparse vectors useful?

Most vocabulary coordinates are zero in one short program. Sparse storage avoids representing every absent term explicitly.

Q49 What is the sigmoid of zero?

0.5. It is the central boundary of the standard logistic function.

Q50 What is the sigmoid of 0.6?

Approximately 0.6457. At threshold 0.5 it predicts AI if that probability is defined for the AI class.

15 mathematical and numerical questions

Q51 For TP=8,FP=3,FN=2,TN=9, what is accuracy?

17/22=0.7727. There are 22 observations and 17 correct decisions.

Q52 For those counts, what is precision?

8/11=0.7273. Use predicted-positive count TP+FP in the denominator.

Q53 For those counts, what is recall?

8/10=0.8. Use actual-positive count TP+FN in the denominator.

Q54 For those counts, what is F1?

16/21=0.7619. Compute 2TP/(2TP+FP+FN).

Q55 For those counts, what is specificity?

9/12=0.75. Compute TN/(TN+FP).

Q56 Scale 4 using training mean 3 and SD 1.

The standardized value is 1. Subtract the training mean and divide by the training SD.

Q57 For x=(2,1),w=(0.7,-0.3),b=-0.5, compute z.

0.6. $0.7\times2-0.3\times1-0.5=0.6$.

Q58 Compute sigmoid(0.6).

Approximately 0.6457. Use $1/(1+\exp(-0.6))$.

Q59 What is log loss for y=1,p=0.6457?

Approximately 0.4375. The positive-label contribution is $-\ln(p)$.

Q60 Compute balanced binary entropy.

1 bit. Each of the two terms is $-0.5\log_2 0.5=0.5$.

Q61 Compute pure-node Gini.

Zero. One class has probability one, so $1-1^2=0$.

Q62 Compute 3:1 binary Gini.

0.375. $1-0.75^2-0.25^2=0.375$.

Q63 Compute smoothed IDF with N=2,df=1.

$1+\ln(3/2)=1.4055$. The implementation uses smoothing and adds one after the logarithm.

Q64 Compute sublinear TF for count 2.

$1+\ln 2\approx 1.6931$. Repeated words grow logarithmically rather than linearly.

Q65 What happens to p=0.5442 at thresholds 0.5 and 0.6?

AI at 0.5 and human at 0.6. The probability does not change and no retraining is needed.

15 implementation questions

Q66 Where is the frontend API address configured?

In the API constant in frontend/src/App.tsx. It currently targets http://localhost:8000/api. A different deployment would require configuration rather than assuming localhost refers to a remote service.

Q67 Why do models and metrics return the same structure?

Both GET routes decorate the same metrics function. It loads the generated metrics.json. This is a small local API design, not a separate model registry.

Q68 Why explicitly select source and numeric columns before prediction?

The fitted transformer expects that schema. Explicit selection prevents group IDs and labels from accidentally becoming classifier inputs.

Q69 What is actually inside codetrace.joblib?

One selected fitted pipeline, its model name, feature names and metadata. It contains learned TF-IDF/scaler state and a classifier, not the raw frontend or three selectable model objects.

Q70 How does the UI render Pydantic validation errors?

It joins msg fields when detail is an array. String detail is also supported. This prevents an object array from producing an unreadable error or breaking React rendering.

Q71 When does React retrieve the model comparison?

The mount effect fetches /api/metrics. It does not retrain or recompute metrics in the browser. The backend returns the actual training output JSON.

Q72 Why is there an OPTIONS request before analysis?

The browser may perform a CORS preflight. The frontend and backend use different ports. The server explicitly allows the two local Vite origins and POST.

Q73 What happens if the saved model file is missing?

predict.analyze raises RuntimeError, which the route maps to HTTP 503. The message tells the user to train the project. Health also exposes model_ready based on artifact existence.

Q74 Is the model cached across API calls?

No; joblib.load runs per request. This is simple but incurs disk/deserialization cost. A startup-loaded model could improve efficiency with explicit reload behavior.

Q75 How does inference avoid confusing probability columns?

It looks up ai in pipeline.classes_. String classes are sorted by the estimator; assuming AI is always column one would be wrong here.

Q76 Can submitting raise RuntimeError execute that exception?

No. It is only parsed and featurized. A regression check submits that text and verifies an ordinary model response without executing the statement.

Q77 What is the purpose of backend/__init__.py?

It makes backend an importable package. The API can use relative imports when launched as uvicorn backend.main:app from the root.

Q78 How are trained model versions traced?

The artifact records schema and library versions and the dataset SHA-256. The technical inventory also hashes the model file. This supports traceability but is not an automatic compatibility enforcement mechanism.

Q79 Why is the Analyze button disabled during a request?

The busy state prevents repeated submissions while that request is pending. Finally resets the state after success or failure, so an error does not leave the interface permanently blocked.

Q80 How can you prove the metrics are not hardcoded UI scores?

Compare GET /api/metrics with models/metrics.json and inspect the React mapping. The HTTP verification does that equality check; the frontend renders object entries rather than literal accuracy constants.

10 reliability and validity questions

Q81 Does the factorial probe prove generalization?

No. It is one unlabeled unseen task, so accuracy cannot be computed.

Q82 Why should we trust these results?

They are reproducible software outputs with documented scope. Trust the measurements on this fixture, not a claim of universal authorship detection.

Q83 Can it detect an unseen code generator?

There is no evidence for that. No provenance-verified generator data or held-out-model protocol is used.

Q84 What should be improved first?

Collect suitable data and define independent evaluation units. Advanced models cannot repair invalid provenance or template leakage.

Q85 What if a human writes a docstring?

The model may raise the AI score. Docstrings are a synthetic class cue here; this is an example of why false accusations are possible.

Q86 What is the main completed contribution?

An explainable educational pipeline and reproducible sensitivity study. Operational reliability remains unvalidated and must not be claimed during the viva.

Q87 Why are library versions recorded?

Serialized estimators can depend on library behavior. The artifact records schema, Python, scikit-learn, joblib and dataset hash for reproducibility.

Q88 What changes require retraining?

Any change to feature semantics or fitted vocabulary. Old scaler parameters and learned weights assume the previous feature definitions.

Q89 How is a model tie resolved?

The first candidate wins after equal validation F1 and accuracy. Logistic Regression appears first in the ordered candidate dictionary.

Q90 Is there a real-world accuracy claim?

No. All performance values refer to the repetitive synthetic dataset or explicitly named experiments.

Section questions for YASH RAJ AGARWAL

These questions relate to slides 1–6.

What does this project actually classify?

Synthetic code-style labels. The generator assigns human and ai strings to templates. Neither label proves actual provenance.

How many records are used?

160 unique examples from 200 rows. Forty exact duplicates are removed after stripping boundary whitespace.

What is group splitting?

Keeping related records in one partition. GroupShuffleSplit keeps each problem_id exclusively in training, validation or test.

Are there 20 independent tasks?

No. The IDs reuse a few filter templates with naming and modulus variants.

Section questions for NISHAANTH S

These questions relate to slides 7–12.

How does tokenization work?

Python tokenize yields lexical tokens. The numeric token statistics include names, operators and literals while excluding comments and layout.

Does nesting depth mean raw AST depth?

No. It increments selected control constructs and function definitions; expression layers alone do not increment it.

What is the smoothed IDF formula?

1+ln((1+N)/(1+df)). The extra ones avoid zero divisions and follow TfidfVectorizer defaults.

Why did you choose XGBoost?

We did not implement it. It belongs to the synopsis and future scope; the actual project uses three scikit-learn baselines.

Section questions for AMAN SHARMA

These questions relate to slides 13–18.

What happens on Analyze?

React posts JSON, FastAPI validates, saved inference returns JSON. The fitted transformer and classifier run; training and source execution do not.

What is feature ablation?

Removing feature groups and retraining. Validation comparisons reveal conditional reliance, though this trivial fixture gives identical class metrics.

Does the factorial probe prove generalization?

No. It is one unlabeled unseen task, so accuracy cannot be computed.

What if a human writes a docstring?

The model may raise the AI score. Docstrings are a synthetic class cue here; this is an example of why false accusations are possible.